

Toxic Comment Classification with an LSTM

Task 2 | Executed experiment on the Jigsaw dataset

This project trains and evaluates a six-output multi-label classifier on the full original Jigsaw training CSV. After normalized-duplicate handling, 159,298 unique comments are split into training, validation and held-out test sets. The configuration selected using validation macro average precision is **weighted bce**. On the held-out test set, validation-tuned thresholds yield micro F1 **0.7082** and macro F1 **0.5709**. These are local held-out results, not a Kaggle leaderboard score.

Dataset and exploratory findings

The downloaded Kaggle train.csv contains 159,571 rows and 8 columns: an ID, English comment text and six binary labels. Labels can co-occur; the six outputs are independent decisions, not mutually exclusive categories. The original data has 89.8% comments with no positive labels and 6.2% with multiple positive labels.

Label	Positive comments	Original prevalence
toxic	15,294	9.584%
severe_toxic	1,595	1.000%
obscene	8,449	5.295%
threat	478	0.300%
insult	7,877	4.936%
identity_hate	1,405	0.880%

Missing text: 0; empty normalized comments removed: 0; duplicated IDs: 0. There were 0 exact repeated texts and 273 repeated normalized texts. In 41 normalized groups, annotations disagreed. We collapsed normalized-identical comments and took the union of their positive labels, retaining the first ID/text. This policy prevents identical normalized inputs from crossing splits, but changes annotation prevalence slightly and can preserve disputed positive labels.

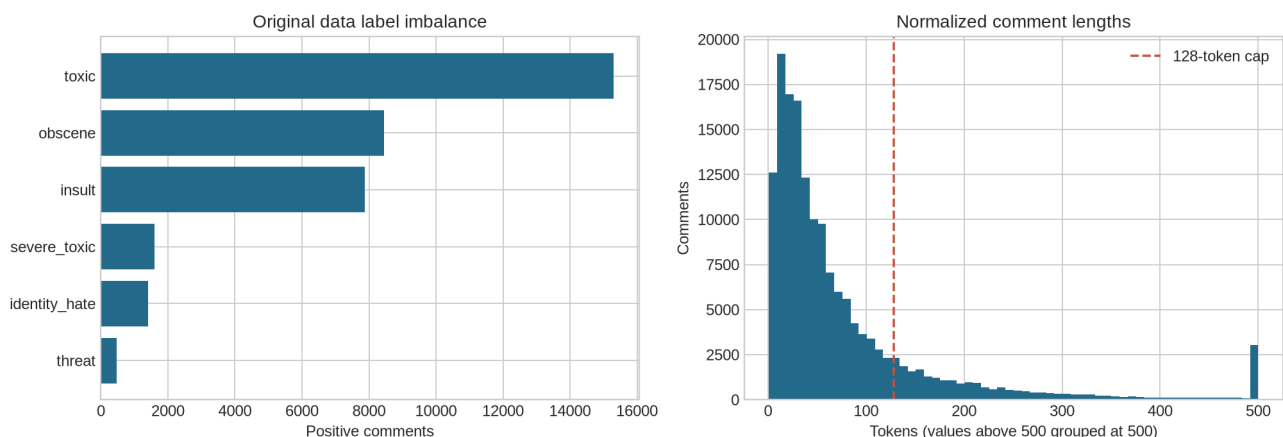


Figure 1. Original label counts and normalized token lengths. The imbalance motivates per-label evaluation and a weighted-loss comparison. The 128-token cap is a compute-budget choice, not a property of the dataset.

Data preparation and leakage controls

Text is HTML-unescaped, lowercased, whitespace-normalized, and URLs/user mentions receive placeholders. Tokenization preserves words, contractions, negation and punctuation. Stopword removal and stemming are omitted to retain meaning and sequential cues. Repeated normalized inputs are collapsed before splitting; no labels from validation or test enter training.

Split	Comments	Purpose
train	127,438	Fit vocabulary, class weights and model parameters
validation	15,930	Select checkpoints, loss configuration and thresholds
test	15,930	Evaluate the frozen selected pipeline

Two iterative multi-label stratified splits approximate 80/10/10 and preserve rare-label marginals; each split is checked for positive examples of every category. Pairwise normalized-text overlap is asserted to be empty. The exact split IDs are saved in `split_assignments.csv`. Near-duplicate paraphrases are not detected.

The vocabulary is fitted only on training text, with at most 30,000 tokens and minimum frequency 2; the realized size is 30,000, including PAD=0 and UNK=1. Unknown tokens map to UNK. Each sequence keeps its first 128 tokens and batches are right-padded dynamically. Empty token lists map to UNK at inference.

Training token-length median: 44; 95th percentile: 277. The cap truncates 16.4% of training, 16.2% of validation and 15.9% of test comments. Validation/test unknown-token rates are 2.7%/2.5%. These are reported limitations rather than hidden preprocessing losses.

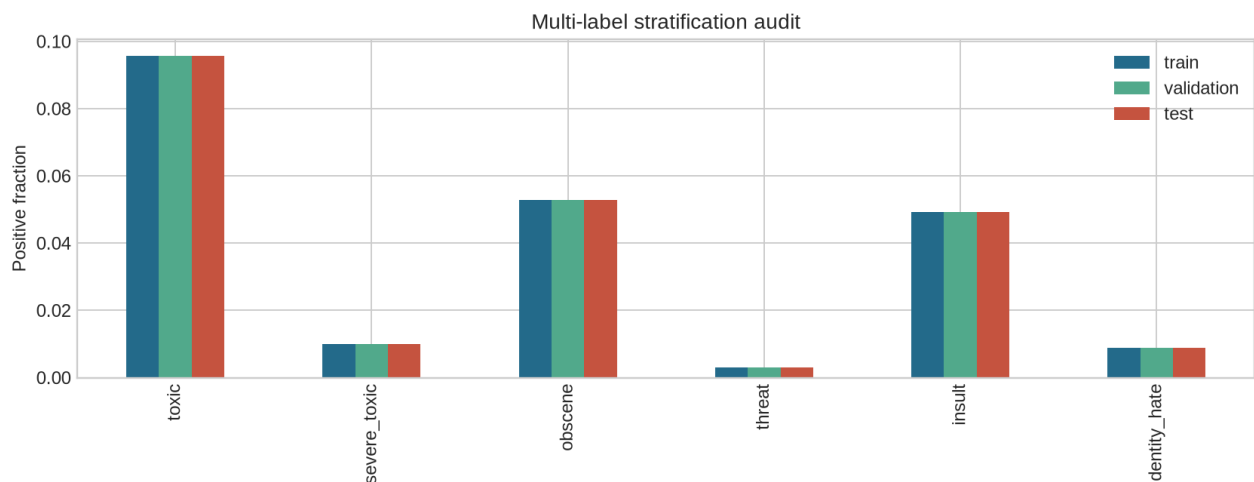


Figure 2. Label prevalence across the three splits. Stratification preserves approximate proportions; it does not make the labels mutually exclusive.

Embedding and LSTM architecture

Token IDs → trainable 64-dimensional embedding → one unidirectional LSTM with 64 hidden units → dropout 0.3 → linear layer with six logits. The model has 1,953,670 trainable parameters. For each sequence, the classifier gathers the LSTM output at its last real token. Because the LSTM is unidirectional, later right-padding cannot alter that earlier state; no padded final position is used.

$$\begin{aligned}i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), & f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c), & c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), & h_t &= o_t \odot \tanh(c_t)\end{aligned}$$

The gates regulate information retention and output. Input matrices are initialized gate-wise with Xavier, recurrent matrices gate-wise orthogonally, and the effective forget-gate bias is one. The embedding starts from a small normal distribution and its padding row stays zero. This is a from-scratch embedding, without pretrained word vectors.

Loss and imbalance experiment

$$\mathcal{L} = -\frac{1}{6N} \sum_{n=1}^N \sum_{c=1}^6 [q_c y_{nc} \log \sigma(z_{nc}) + (1 - y_{nc}) \log(1 - \sigma(z_{nc}))]$$

Unweighted BCE uses $q_c = 1$. Weighted BCE uses $q_c = N_{\text{negative},c} / N_{\text{positive},c}$, calculated on training only. BCEWithLogitsLoss combines sigmoid and BCE for numerical stability; sigmoid is applied explicitly when converting logits to probabilities. No softmax is used. Weighting increases the influence of missed positives and can improve recall at the cost of false positives; focal loss is an alternative not run in this comparison.

Category	Train positive weight
toxic	9.44
severe_toxic	99.03
obscene	17.90
threat	333.48
insult	19.27
identity_hate	112.78

Training protocol

Both loss configurations use the same seed (42), architecture and data split. AdamW uses learning rate 0.002, weight decay 0.0001, batch size 256, gradient clipping at 1.0, and up to 6 epochs. A plateau scheduler is configured to halve the learning rate; early stopping uses validation macro average precision with patience 2. Execution used cpu and PyTorch 2.14.1+cpu.

Validation comparison and model selection

Configuration	Best epoch	Macro AP	F1 at 0.5	F1 tuned
unweighted bce	5	0.5384	0.4310	0.5427
weighted bce	4	0.5537	0.4541	0.5592

The selected configuration is weighted bce because it has the highest validation macro average precision. AP averages precision at recall increments; it is not identical to the trapezoidal PR-AUC also reported. Macro averaging gives each toxicity category equal weight. Loss magnitudes are comparable within a configuration, but weighted and unweighted BCE have different scales.

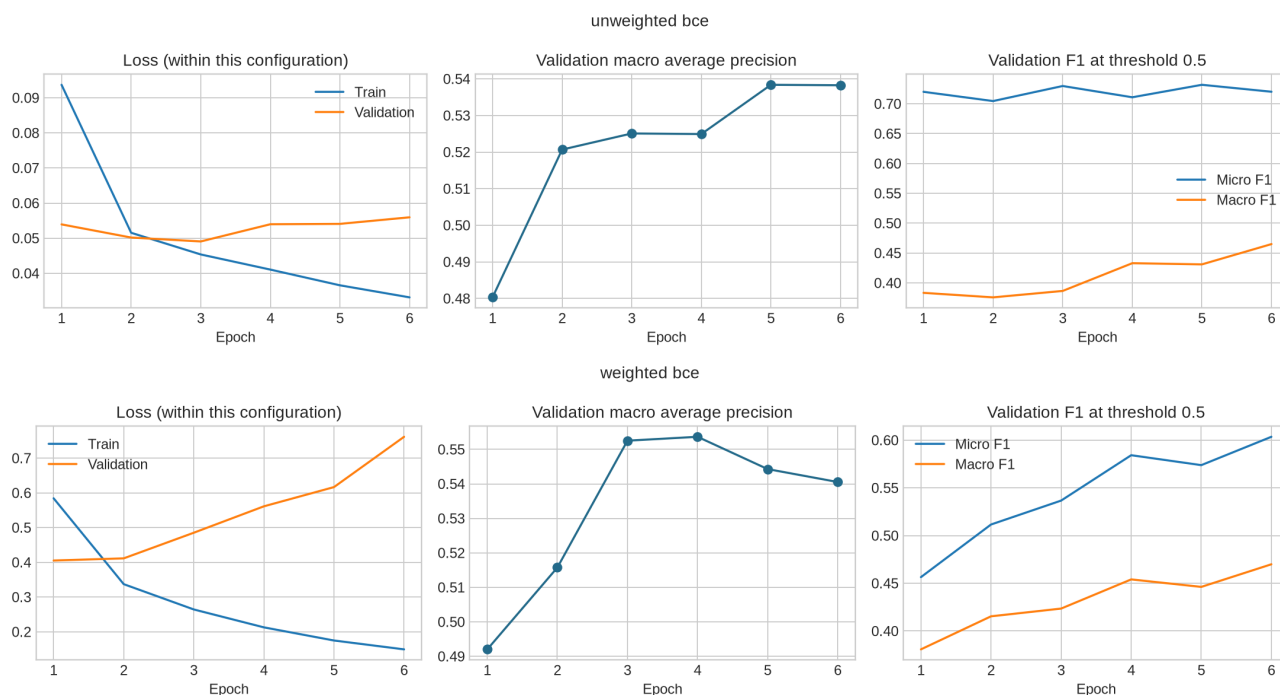


Figure 3. Actual training/validation loss, validation macro AP and F1 at threshold 0.5. Checkpoints are restored at the best validation epoch. Weighted BCE improves its ranking metric through epoch 4, while later train loss falls and validation macro AP declines. This supports retaining the earlier checkpoint. Rising validation loss can also reflect overconfidence or a metric mismatch.

Decision thresholds

For each category, a fixed grid from 0.05 to 0.95 is searched on validation to maximize that category's F1. Ties prefer values closest to 0.5. These thresholds are frozen before test evaluation. Validation F1 after this search is optimistic; held-out test performance is the independent estimate.

Label	toxic	severe_toxic	obscene	threat	insult	identity_hate
Threshold	0.82	0.95	0.93	0.95	0.88	0.95

Held out test results

Label	Precision	Recall	F1	ROC AUC	PR AUC	Support
toxic	0.795	0.768	0.781	0.962	0.854	1525
severe_toxic	0.342	0.761	0.472	0.989	0.410	159
obscene	0.796	0.798	0.797	0.988	0.876	843
threat	0.214	0.688	0.327	0.983	0.318	48
insult	0.626	0.790	0.699	0.978	0.750	786
identity_hate	0.260	0.536	0.350	0.968	0.245	140

Threshold policy	Micro F1	Macro F1	Macro ROC AUC	Macro PR AUC
Fixed 0.5	0.5854	0.4613	0.9779	0.5753

Validation tuned 0.7082 0.5709 0.9779 0.5753

Macro AP is 0.5790; exact-match accuracy is 0.9075; Hamming loss is 0.0232. ROC-AUC and PR-AUC use probabilities and do not change when the threshold changes. An all-negative baseline reaches exact-match accuracy 0.8981 but has micro/macro F1 of zero. This demonstrates why the many all-zero comments make accuracy alone misleading.

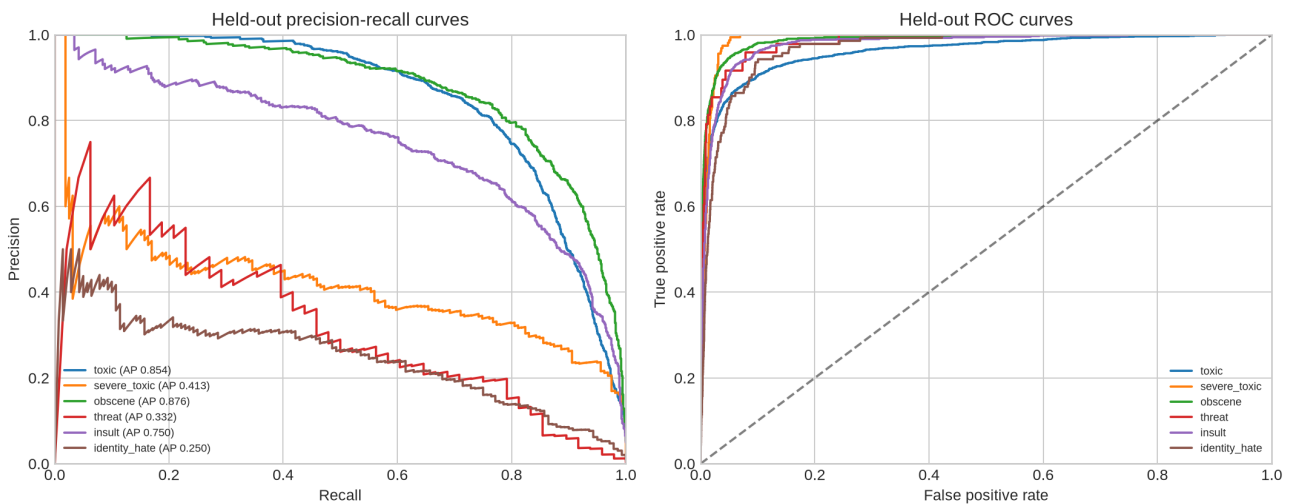


Figure 4. Test precision-recall and ROC curves. AP values in the legend are separately calculated average precision, whereas the table reports trapezoidal PR-AUC. Positive support is small for rare categories, so those estimates are more uncertain.

Interpreting category performance

The strongest category by test F1 is obscene (0.797); the weakest is threat (0.327). This is a measured ranking for this run. The imbalance and smaller support make rare-label learning and threshold selection harder. A high ROC-AUC can coexist with modest F1 or precision, so it is not sufficient evidence of useful moderation performance.

Error analysis and limitations

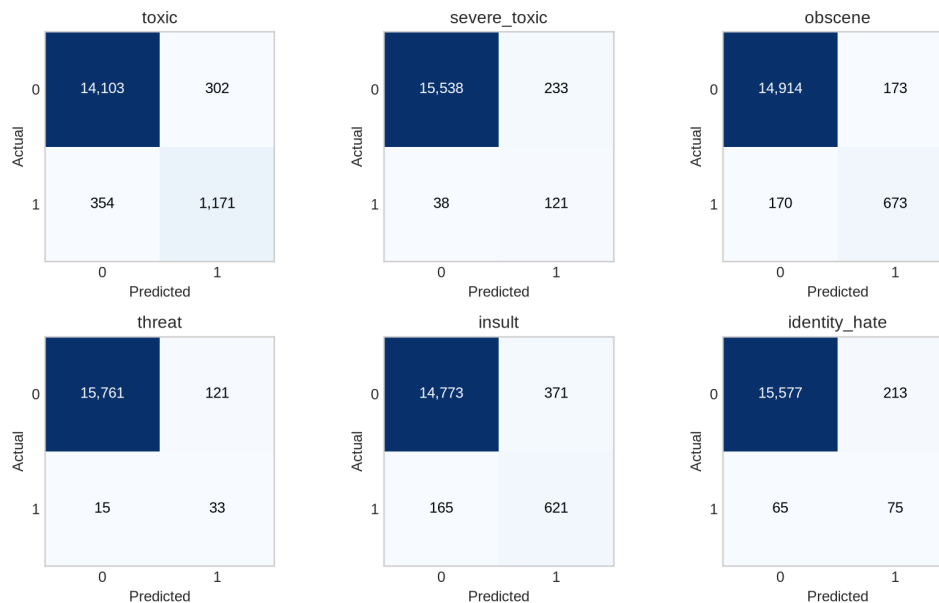


Figure 5. Six confusion matrices at frozen validation-tuned thresholds. Rows are actual labels and columns are predictions; the cells show TN, FP, FN and TP.

1,474 of 15,930 test comments have at least one label error. The any-label error fraction is 6.6% among truncated comments and 9.8% among non-truncated comments. This is an association, not evidence that truncation alone caused the difference. `error_examples.csv` records up to five confident false positives and five false negatives per label, with ID, score, threshold, length and text excerpt.

Inspection of one confident false positive and false negative per category found a missed indirect wish of harm in threat, misspelled profanity in severe_toxic, and errors in long truncated comments. Several apparent false positives contain plausibly relevant language, indicating possible annotation ambiguity rather than proving a wrong annotation. These sampled observations guide further investigation; they do not isolate the model's causal failure mechanism.

Strengths. The model learns word representations jointly with the classifier, uses word order, supports overlapping labels, and is small enough for CPU training. Saved preprocessing and thresholds make inference reproducible. Weighted and unweighted training provide a direct check of one imbalance strategy.

Limitations. A single seed and split do not measure run-to-run uncertainty. The last-state representation can lose information in long comments, while the 128-token prefix can discard later toxicity. Word-level unknown tokens and misspellings limit robustness. Wikipedia English differs from other languages and platforms, and annotation disagreement can affect both learning and apparent errors. Sequential computation is slower to parallelize than modern attention-based encoders. Identity terms may be correlated with toxicity without being toxic themselves; this experiment does not measure demographic subgroup fairness.

Reproducibility

The submission includes the executed notebook, training source, tests, checkpoints, training logs, exact split IDs, configuration, environment versions, saved probabilities and metric tables. Re-running the training entry point reproduces the full procedure; minor numeric variation is possible across hardware/library versions. The source CSV SHA-256 is recorded in `data_summary.json`.